PA1 Report: Union-Find Empirical Study

Track A: implementation and empirical study

Student name: Hoang Hung Nguyen
Student number: 14682330

October 2026

Repository: https://github.com/AldrichSolan26/union-find-study

Video walkthrough: https://youtu.be/SqfRJ7w-RLw

(a) What I built

Union-Find (the disjoint-set data structure) maintains a partition of the elements {0, …, n−1} into disjoint sets. `unite(a, b)` merges the sets containing a and b, and `find(x)` returns a canonical representative, the *root*, of x's set, so `connected(a, b)` is simply `find(a) == find(b)`. Each set is stored as a tree of parent pointers, and the cost of a find is the length of the path from x up to its root. Two independent optimisations keep those paths short: **balanced linking** (how two roots are joined) and **path compression** (what find does to the path it walks).

I implemented it in C++17 with the linking and find strategies as separate template policies, so one class produces all 3 × 4 = 12 combinations.

Policy	Options	Bound (per operation)
Linking	**naive**: hang the first root under the second **rank**: union by rank (1 byte per element) **size**: union by size (4 bytes per element)	naive: Θ(n) per find in the worst case rank or size alone: O(log n)
Find	**none**: walk to the root **full**: two-pass full compression **halving**: every other node → grandparent **splitting**: every node → grandparent	compression with naive linking: O(log n) amortised rank/size + any compression: O(α(n)) amortised, α(n) ≤ 4 in practice

Key design decisions.

- **Flat arrays.** `parent` is a `std::vector<int>` indexed by element, not node objects with pointers: compact, cache-friendly, and the same layout for every variant.
- **Iterative find.** Every find is a loop. A recursive find needs one stack frame per node and overflows on a naive-linked chain; the tests run finds on a 2^{20}-node chain.
- **Rank in one byte.** Rank never exceeds $\log_2 n$, which is below 32 for any `int`-sized n, so `std::uint8_t` is enough. Size needs a full `int`.
- **Policies as templates.** No virtual calls in the hot path, so timing differences reflect the algorithms rather than dispatch.
- **Zero-cost instrumentation.** A third template parameter, `Stats`, is either `NoStats` (empty inline functions that compile away; used for every timed run) or `CountStats`, which counts parent-array reads and writes and the path length of every find. This let me measure *work* and *time* separately.
- **Defensive interface.** `find`, `unite` and `connected` throw `std::out_of_range` for an invalid element; n = 0 is allowed and n < 0 throws. The check is one well-predicted comparison, the same for every variant.
- **Reproducible workloads.** `std::uniform_int_distribution` and `std::shuffle` produce different sequences on different standard libraries, so bounded random numbers and shuffling are written out by hand on top of `std::mt19937_64`. A clang build using Microsoft's standard library reproduced all 1,080 compared rows of checksums and work counts exactly.

Navigating the code.

- `include/union_find.hpp`: the structure and all link, find and stats policies; the core invariant (parent[x] = x exactly for roots, and the parent graph has no cycles) is documented at the top, with where it is established and relied on.
- `include/variants.hpp`: compile-time list of the 12 variants.
- `tests/test_union_find.cpp`: 7,315 correctness checks (below).
- `bench/workloads.hpp`, `bench/harness.hpp`, `bench/bench.cpp`: workload generators, the timed code, and the timing harness.
- `scripts/run_study.py`, `scripts/plot.py`: run the whole study; turn the CSV into figures and tables.
- `experiments/`: three small programs, each testing one explanation from section (b).

Testing. The tests compare every variant with a brute-force oracle on random operations and check the structural invariants after each batch: no cycles; rank[parent[x]] > rank[x]; a root of rank r has at least 2^r elements; size[root] equals the true set size. They also check the $\log_2 n$ height bound (and that the binomial workload reaches it exactly), the exact parent pointers each find policy leaves after one find on a chain, exact read and write counts, and Kruskal's MST weight against an independent Prim's algorithm. To test the tests, I injected eight bugs one at a time, for example deleting the `ra == rb` check or making halving step to the parent instead of the grandparent. All eight were caught. Two of them still gave correct answers and were caught only by the exact-shape tests: they were performance bugs, not correctness bugs.

(b) Empirical study

The question. The textbook analysis says that with balanced linking and path compression every operation costs amortised O(α(n)), which is effectively constant. Does that predict measured time? Which optimisations matter in practice? And where theory and practice diverge, why?

What I compared

All 12 variants on six workloads, for every power of two from $n = 2^{10}$ to 2^{24}, each with three seeds: 3,078 measurements in total. Naive linking with no compression is the baseline. Every measurement records both **time** (nanoseconds per operation) and **work**: parent-array reads and writes per operation, mean path length per find, and final tree depth. Work is counted exactly and does not depend on the machine, so the two can be compared directly.

Hypotheses

H1–H7 were written and committed to the repository before the full benchmark run (commit `5f9148f`), informed only by a small smoke test. H8 and H9 were added after measurement problems appeared during the first full runs, and are marked as not pre-registered.

#	Hypothesis	Held?	Key number
H1	Work follows the theory: naive+none is Θ(n) per find on chain, rank/size+none is Θ(log n) on binomial, and rank/size + compression is flat	**Yes**	path lengths match the exact predictions; 4.00003 vs 4.00000 at $n = 2^{24}$
H2	Time does not follow the theory: with work flat, ns/op still rises > 2× from 2^{10} to 2^{24}, mostly where the arrays outgrow L2 and L3	**Yes**	reads/op ×1.01; ns/op ×3.0–3.7
H3	Locality explains much of H2: local-shuffled is ≥ 2× slower than local at 2^{24}	**No**	1.33× (range 1.31–1.44)
H4	The choice of compression matters much less than having some; one-pass is at least as fast as two-pass	**Partly**	halving fastest in 149/180 configurations; splitting slower than full in 119/180

#	Hypothesis	Held?	Key number
H5	Rank beats size at large n, because its array is smaller	**Partly**	size up to 27% slower at 2^{21}–2^{23}, but at 2^{24} only on kruskal (9–15%); at small n size is 3–11% faster
H6	naive + compression is < 3× slower than rank + compression on random	**Yes**	max 2.78×, still rising at 2^{24}
H7	Without the `volatile` sink, the compiler deletes finds and naive+none looks fastest	**Yes**	2.2 vs 15,486 ns/op (chain, 2^{15})
H8*	Replaying one op sequence for every rep lets the branch predictor learn it	**Yes**	5.5 vs 19.1 ns/op (random, 2^{10})
H9*	Windows power throttling makes results depend on the order variants run in	**Yes**	2.27–2.31× slower after sustained load

* Added after the first full runs; not pre-registered.

Data generation

Workload	Operations (n elements)	Purpose
random	2n: half `unite`, half `connected`, uniform endpoints	the typical case
chain	`unite(i, i+1)` for all i, then n uniform finds	worst case for naive linking: a path of length n − 1
binomial	`unite(i, i+s)` for s = 1, 2, 4, …, then n uniform finds	worst case for rank/size: every merge is a tie, so the height is exactly $\log_2 n$
local	like random, but b is within 16 of a	cache-friendly pairs
local-shuffled	local with element ids relabelled by a random permutation	identical work to local (tested exactly), scattered addresses
kruskal	Kruskal's MST on 4n random edges plus a heavy spanning path; edges pre-sorted	a real algorithm built on Union-Find

All generation, including sorting the Kruskal edges, happens before timing starts, so only Union-Find work is measured.

Measurement method

- **Machine.** Intel Core i9-14900HX (a hybrid CPU with performance and efficiency cores), 31.7 GB RAM, Windows 11 (build 26300), Balanced power plan, on AC power. Every timed run was pinned to logical CPU 4, a performance core with 48 KB L1d, 2 MB L2 and a shared 36 MB L3.
- **Build.** g++ 16.2.0 (MSYS2) with `-O3 -DNDEBUG`. The code was also built and tested with clang 23 and with CMake 4.4.
- **Per (workload, n, seed, variant):** one untimed counting run (`CountStats`), which gives the work numbers; one untimed warm-up; then timed runs, each on a freshly constructed structure (construction not timed), until at least 3 runs and 100 ms of timed work (at most 51 runs). I report the median. The plots show the median of the three seeds, with a min–max band.
- **Fresh sequences.** When one op sequence is short (under 2^{18} operations), the timed runs cycle through several freshly generated sequences of the same workload instead of replaying one; otherwise the branch predictor learns it (H8).
- **Safeguards.** The harness opts out of Windows power throttling (H9), and every find result is folded into a `volatile` sink so the compiler cannot delete it (H7). Each safeguard can be switched off with a flag to reproduce the failure.
- **Checksums.** Every run returns a checksum (merges plus "yes" answers, or the MST weight). All variants agreed on all 3,078 rows; a mismatch stops the run.

- **Drift monitor.** A fixed small benchmark, the "canary", ran before every chunk of the study. Across the final dataset it stayed between 20.60 and 20.77 ns/op (0.8%). The median spread between timed runs of one measurement was 1.0%, and between seeds 0.7%.
- **Cap.** naive+none is measured only up to n = 2^{15}, because on chain it takes seconds per run beyond that.

Results

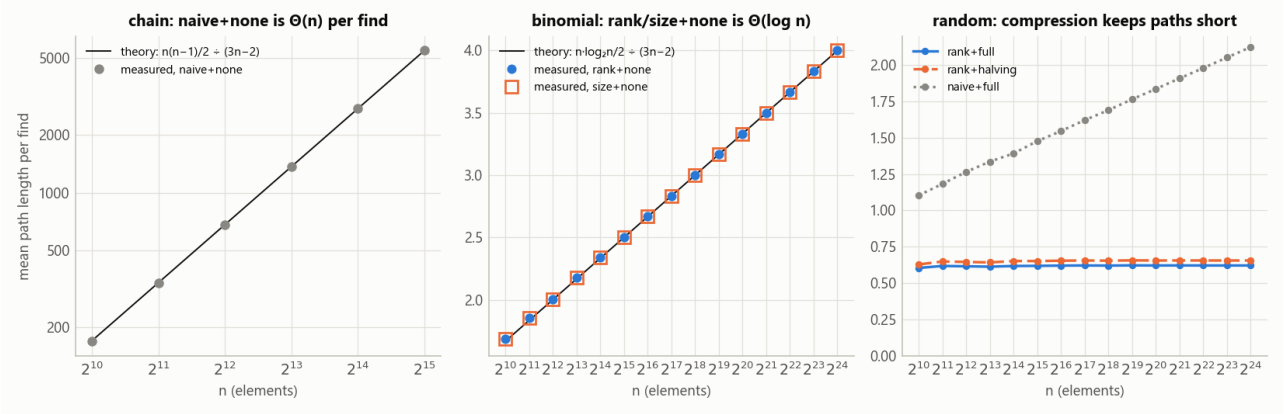


Figure 1. Measured mean path length per find against the textbook prediction. The lines are exact expectations derived from how each workload is built, not fits.

The theory predicts the work exactly. naive+none on chain grows linearly (169.4 at 2^{10} against 170.6 predicted; 5,475.5 at 2^{15} against 5,461.3). rank/size+none on binomial reaches exactly $\log_2 n \div 6$ (4.00003 at 2^{24}). With balanced linking and compression, the random-workload path stays at 0.60–0.66 from 2^{10} to 2^{24}.

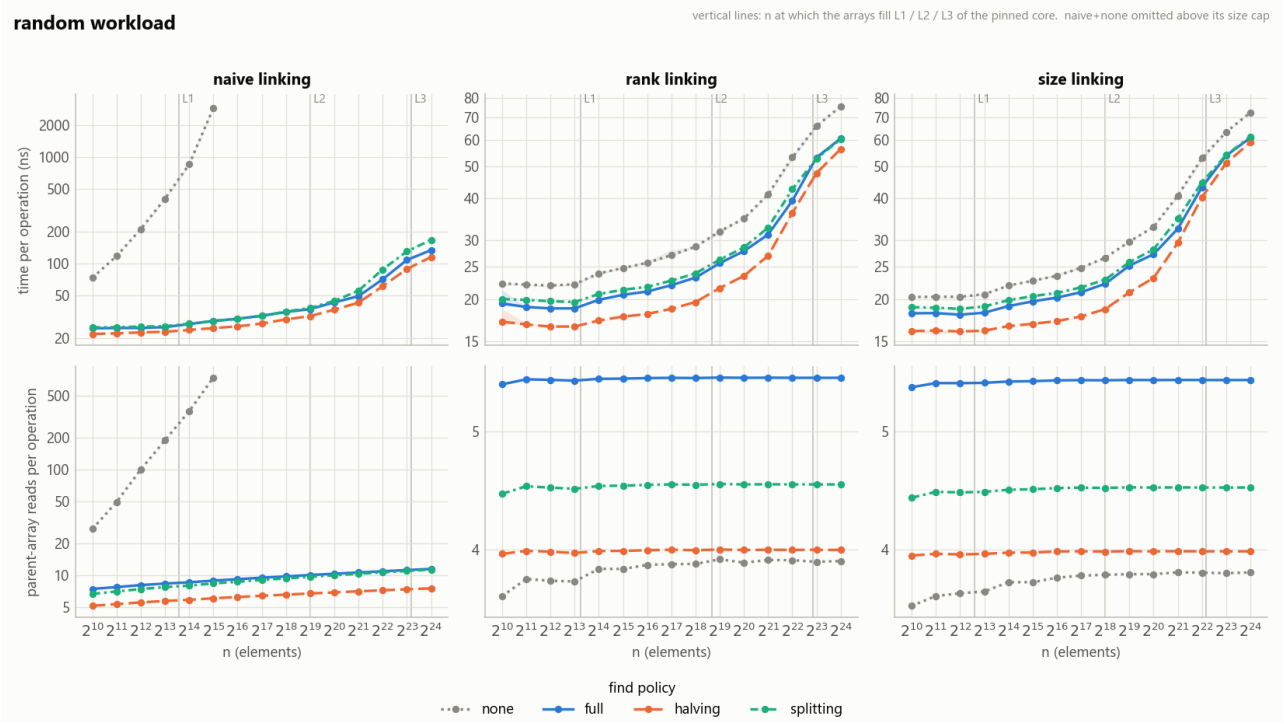


Figure 2. Random workload: time per operation (top row) and parent-array reads per operation (bottom row), one panel per linking policy. Vertical lines mark where the arrays fill each cache level of the pinned core.

Reads per operation are flat for every rank/size variant, but time triples: rank+full costs 19.4 ns/op at 2^{10} and 60.5 ns/op at 2^{24}. Halving is the fastest find policy in every panel.

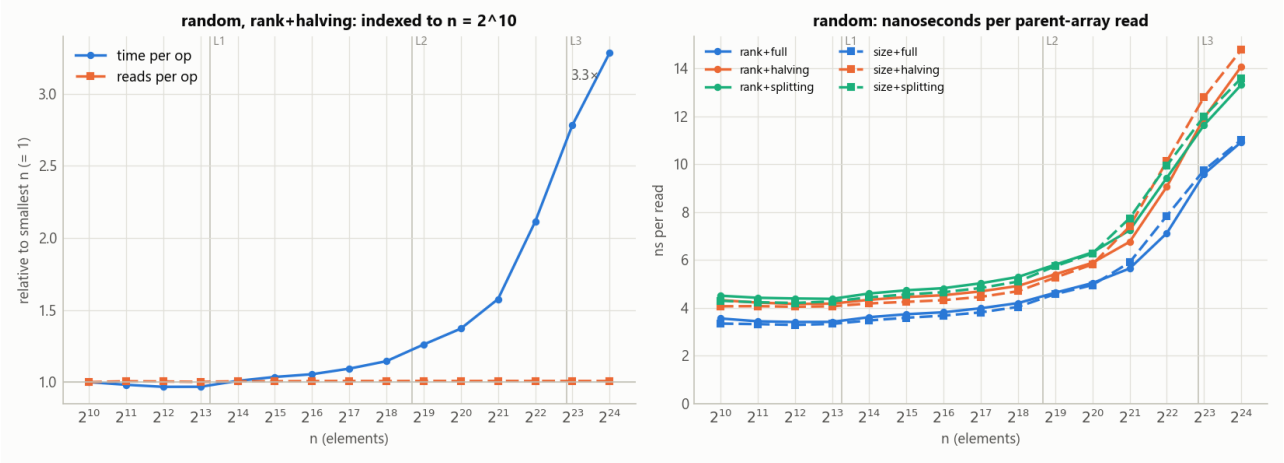


Figure 3. Left: time and reads per operation for rank+halving, both indexed to n = 2^{10}. Right: nanoseconds per parent-array read. Work stays at 1.0× while time reaches 3.3×. The cost of one read rises from 3.4 ns while the arrays fit in L1 to 10.9 ns once they exceed L3 (rank+full), with the steepest rise between 2^{21} and 2^{23}.

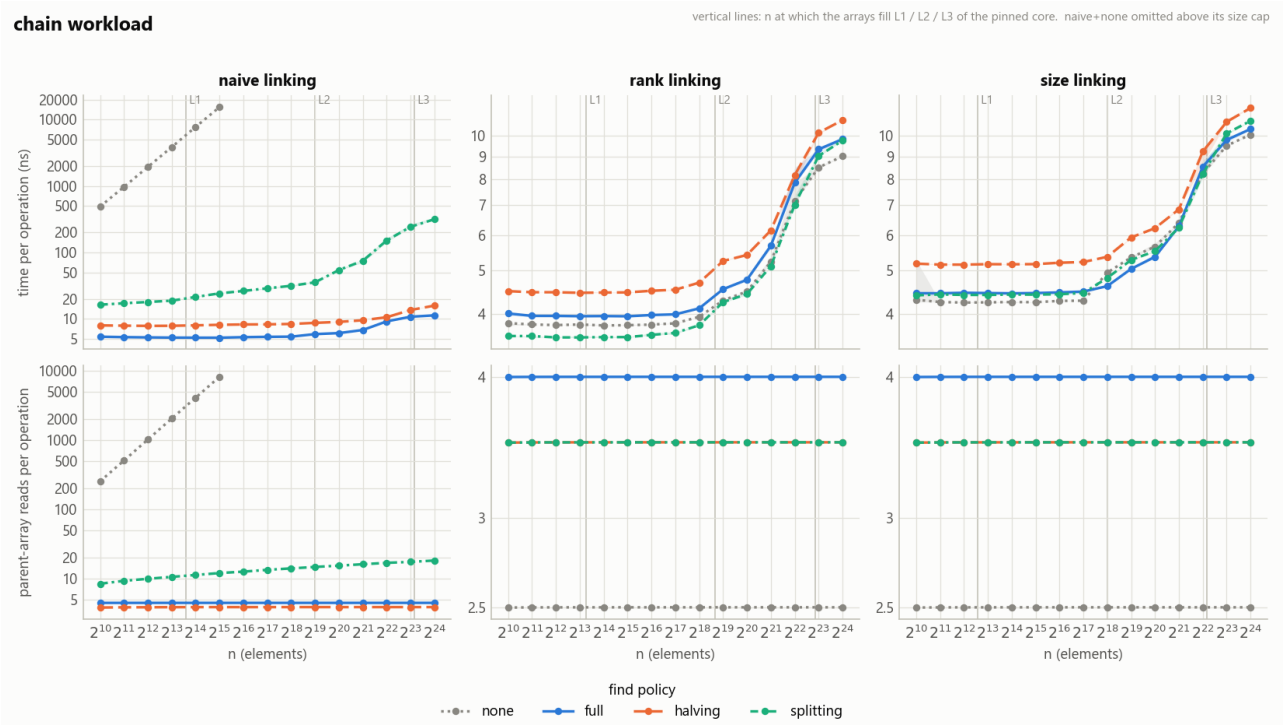


Figure 4. Chain workload, the worst case for naive linking (log-scale time).

naive+none costs 15,493 ns/op at 2^{15}, with a tree of depth 32,767. Rank and size linking turn the same input into a star of depth 1, so every rank/size variant stays under 12 ns/op up to 2^{24}. Here compression only adds work: rank+none is the fastest variant at 2^{24} (9.0 ns/op).

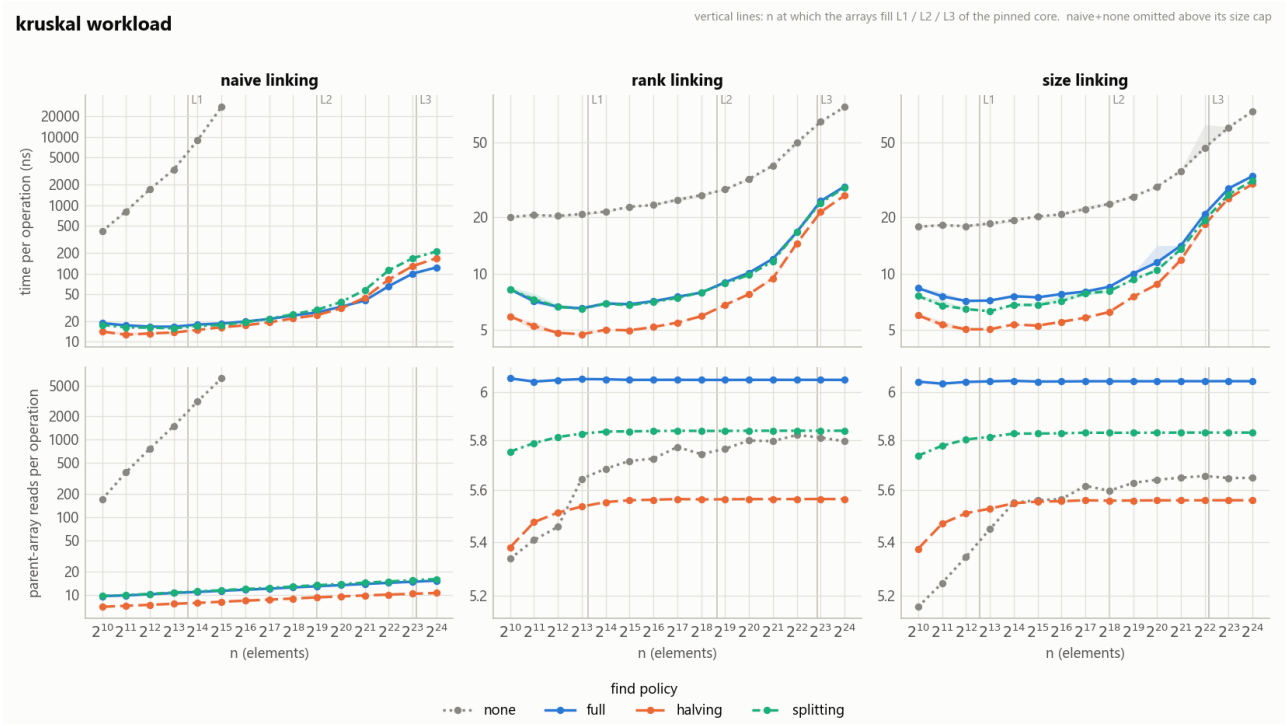


Figure 5. Kruskal workload.

Compression matters most here. rank+none is 2.4–4.6× slower than the best compressed variant, even at $n = 2^{10}$, where everything fits in L1, and even though it does fewer reads (5.3 against 6.1 per operation).

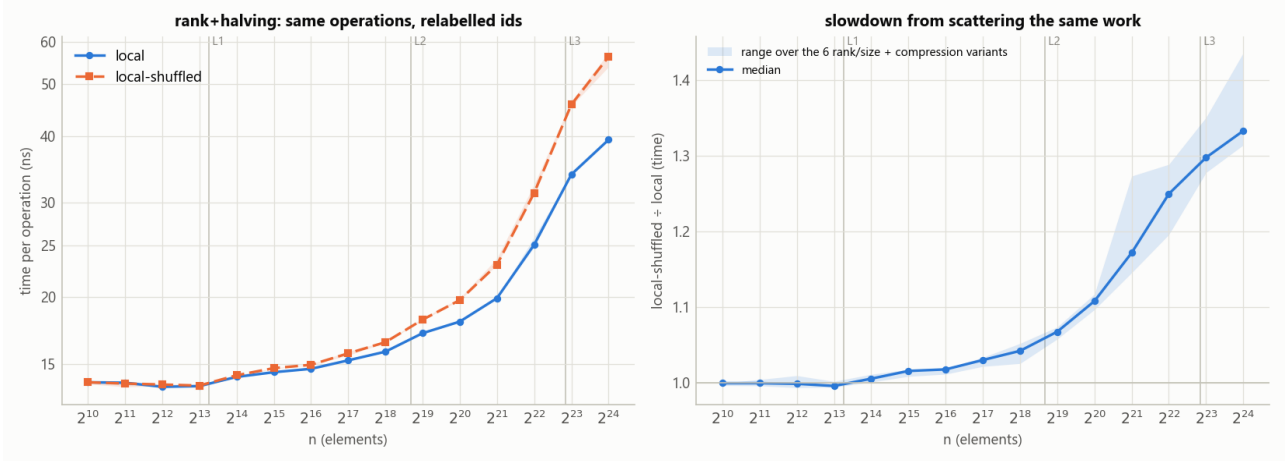


Figure 6. Left: rank+halving on local and local-shuffled, which perform identical operations under a relabelling. Right: their time ratio across the six rank/size + compression variants.

Scattering the second endpoint costs only 1.33× at 2^{24}, and even local itself slows down 2.8× between 2^{10} and 2^{24}.

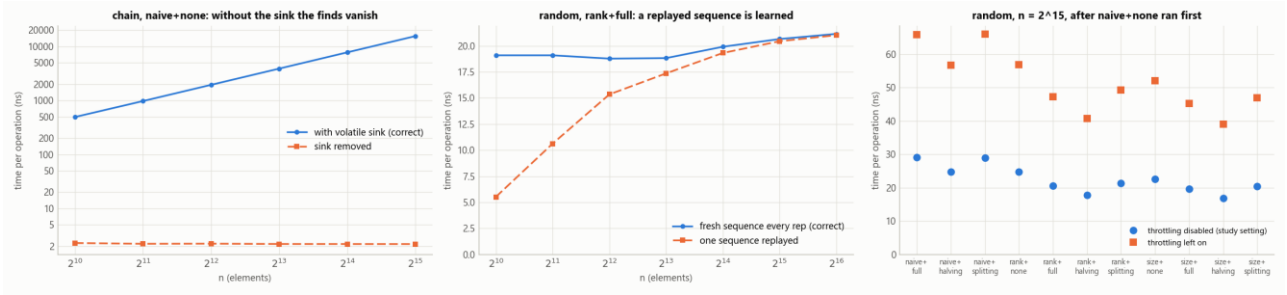


Figure 7. The three measurement pitfalls, each reproduced by switching its safeguard off.

Without the sink, naive+none on chain reports 2.2 ns/op instead of 15,486. Replaying one sequence makes rank+full look 3.5× faster at 2^{10}. With power throttling left on, every variant measured after naive+none is 2.3× slower.

Analysis

H1: supported. Every worst case behaves exactly as analysed (Figure 1). The one case where the theory gives only an upper bound also fits it: naive linking with full compression grows from 1.10 to 2.12 steps per find over fourteen doublings of n, about +0.07 per doubling, which is logarithmic and well under the O(log n) amortised bound.

H2: supported. With the operation count fixed, the source of the data changes. For rank+full on random:

n	Arrays fit in	ns/op	ns per read
2^{10} – 2^{13}	L1 (48 KB)	18.8 – 19.4	3.4 – 3.6
2^{14} – 2^{18}	L2 (2 MB)	19.9 → 23.2	3.6 → 4.2
2^{19} – 2^{22}	L3 (36 MB)	25.6 → 39.4	4.6 → 7.1
2^{23} – 2^{24}	main memory	53.1 → 60.5	9.6 → 10.9

Two details are worth explaining (both inferred). At 2^{24} the cost is lower than one cache miss per read would suggest: 60.5 ns is about 41 ns above the in-cache cost for about 5.5 reads. Many of those reads re-touch lines that were just loaded (the root, and full compression's second pass), and the two finds in each operation are independent, so the CPU can wait on several misses at once. At 2^{10} the cost is not memory at all: 19 ns for 5.5 L1-resident reads is several times L1 latency. H8 suggests the rest is branch mispredictions, because when a replayed sequence lets the branches be learned, the same work costs 5.5 ns instead of 19.1, so about 71% of the small-n cost disappears. "Constant time per operation" is therefore constant at roughly 19 ns in cache and 60 ns out of it.

H3: refuted, because of a flaw in my experiment. local and local-shuffled perform identical work, and the predicted "within 10% while the arrays fit in L2" held (≤ 1.04 up to 2^{18}). But the ratio reached only 1.33 at 2^{24}. The local workload only places b near a *within* one operation; a is still uniformly random *across* operations. So even local jumps to a random cache line on every operation, and its own time grows 2.8× (13.9 → 39.4 ns/op for rank+halving), against 4.1× for local-shuffled. Most of the growth in H2 comes from where successive operations land, not from pairing a with b. A workload with sequential a would be the real locality test.

H4: partly supported. Path halving was the fastest find policy in 149 of 180 (workload, linking, n) configurations; every exception but one is on chain, where rank/size linking already builds a star. Halving does the fewest reads and writes (4.0 reads per operation on random, against 4.5 for splitting and 5.5 for full): it re-points every other node, splitting re-points every node, and full compression walks the path twice. But splitting is *slower* than full compression in 119 of 180 configurations, so "one-pass beats two-pass" holds only for halving. The spread among compression policies stayed under 25% on random (4–23%) but reached 42% on kruskal and 44% on binomial. At 2^{24} it shrinks to 2–13% on every workload except binomial (22–25%), consistent with the extra reads hitting lines already in cache (inferred). Compression's overall benefit depends on the workload: 2.4–4.6×

on kruskal, 1.8–5.8× on binomial, but only 1.23–1.53× on random, where rank linking alone already keeps paths short (0.96 at 2^{24}).

A related, measured result: **fewer reads did not mean faster.** On kruskal, rank+none does fewer reads than rank+full but is 2.4× slower at 2^{10}, entirely in L1. On random, rank+none and size+none do the fewest reads of any variant (3.6–3.9 per operation) yet are slower than every rank/size + compression variant at every n. What differs is path length (1.67 against 0.93 steps per find on kruskal). My inferred explanation: each hop must wait for the previous one, and a varying number of hops makes the loop exit hard to predict. Read counts measure work, not latency.

H5: partly supported. The two linking rules build equally shallow forests (reads within 2%). Rank's smaller array does show, but only where the two footprints straddle the L3 size. From 2^{21} to 2^{23} size was slower on random, local-shuffled and kruskal at every n, by up to 11%, 20% and 27% respectively, with the largest gap at 2^{22}, where size's arrays (8 bytes per element, 32 MB) nearly fill the 36 MB L3 and rank's (5 bytes, 20 MB) do not. At 2^{24}, where both are far beyond L3, the gap closed to within 5% on random but stayed at 9–15% on kruskal, and I cannot fully explain that difference. At small n the opposite happened: size was 3–11% faster on random and local-shuffled up to 2^{18}. A side experiment (`experiments/rank_branch.cpp`) measured the cause. Rank's `if (rank[a] == rank[b]) ++rank[a]` is an unpredictable branch, because ties are common, while size's `size[a] += size[b]` has no branch. Writing the rank increment branch-free closes about half the gap at $n \le 2^{12}$ and nearly all of it from 2^{14} (at 2^{16}: rank 16.0, size 15.2, branch-free rank 15.3 ns/op).

H6: supported, with a caveat. naive + compression was 1.26–1.28× slower than rank + compression at 2^{10} and up to 2.78× (splitting) at 2^{24}: under 3× everywhere, as predicted. But the ratio is still rising at 2^{24}, because naive's work grows logarithmically (H1), so "under 3×" describes $n \le 2^{24}$, not a bound. In cache the time ratio is smaller than the reads ratio; past L3 it is larger, because naive's extra reads become cache misses.

H7–H9: the measurement had to be debugged. Each pitfall changed results by 2× or more (Figure 7), and each was found only because a number disagreed with expectations. H8 showed up as rank+full getting 3× slower between 2^{10} and 2^{12}, although both fit in L1. H9 showed up as the same measurement reading 21 or 38 ns/op depending on whether naive+none had run first. A 5-second test confirmed the cause: after sustained load, Windows clocked the core down and kept it down, and opting the process out of power throttling removed the effect (18.6–19.7 ns/op throughout). H7 struck twice: the harness guarded against it from the start, but a helper I later wrote for the side experiments discarded a checksum, and rank+none appeared 35% faster than the main benchmark measured, until the two disagreed.

Threats to validity

- **One machine.** Work counts are portable (verified with a second compiler and standard library); times are not. A different CPU moves the cache steps and changes small-n branch costs.
- **No hardware counters.** `perf` is not available on Windows, so cache-miss and branch-misprediction explanations are inferred from controlled experiments (H3, H8 and `rank_branch`), not counted.
- **Code generation at small n.** The same library code on equivalent sequences ran about 10% faster in the side-experiment programs than in `uf_bench` (e.g. 17.7 against 19.4 ns/op), because the timed loop is inlined into a different caller. Comparisons within one program are consistent; small-n differences under about 10% should not be over-read.
- **The drift monitor only sees clock speed.** The canary fits in L2, so memory-bandwidth contention from other programs goes undetected. One chunk was 13–25% slow at 2^{19}–2^{22} with a normal canary; it was caught because the seeds disagreed, and was re-measured.
- **Interrupted runs.** The first full run was split by an 11-hour laptop sleep, and its results were noisier. Every chunk in the final dataset was re-measured in one continuous session; the discarded data and the reasons for each case are preserved in the repository's history (commit `0d9841c`). The re-measured work counts were identical to the first session's, and times agreed to a median of 2%.

- **Timer resolution and tiny runs.** The clock ticks every 100 ns, and at $n = 2^{10}$ a run lasts about 40 µs. 17 of 1,026 (workload, variant, n) points have more than 10% spread across seeds, mostly at $n \le 2^{11}$; the plots use the median of three seeds.
- **Workload realism.** The random workloads are uniform, which is easy for balanced linking. The local workload controls only within-operation locality (H3).

(c) What I learned

"Constant time" is not constant in nanoseconds. I expected flat ns/op for rank + compression because $\alpha(n) \le 4$. The work was flat (rank+full: 5.47 → 5.54 reads per operation from 2^{10} to 2^{24}), but the time went from 19 to 60 ns. The same instructions cost three times as much once the parent array no longer fits in cache. I now read an amortised bound as a count of operations, not a prediction of time.

My benchmark measured the wrong thing three times, and each time the only clue was a number that did not make sense.

1. Small-n timings got 3× slower between $n = 2^{10}$ and 2^{12}, although both sizes fit in L1. The harness replayed the same sequence on every rep, and the CPU's branch predictor had learned it.
2. The same measurement read 21 or 38 ns/op depending on whether naive+none had run before it. That was Windows power throttling.
3. A side experiment showed rank+none 35% faster than the main benchmark did. Its helper discarded the checksum, so the compiler deleted the queries: the exact pitfall the main harness already guarded against.

What I take from this: a benchmark needs its own tests. That means sanity checks against expected behaviour, cross-checks between independent programs, and a switch to turn each safeguard off and confirm that it matters.

Fewer memory reads does not mean faster. On Kruskal, rank+none did fewer reads than rank+full (5.3 against 6.1 per operation) but was 2.4× slower, even at $n = 2^{10}$. What mattered was the length of each dependent chain of pointer hops (1.67 against 0.93 per find), not the number of reads.

My locality experiment did not test what I thought it tested. I predicted that scattering the second endpoint would make local-shuffled at least 2× slower; it was 1.33×. My workload kept the first endpoint random across operations, so it only tested locality within one operation. I learned that a hypothesis can fail because the experiment is wrong, not because the idea is.

What I understand now that I did not before. Why union by rank bounds the height by $\log_2 n$: a tree only gets taller when two trees of equal rank merge, so reaching rank r needs at least 2^r elements. How rank and size differ: rank changes only on a tie and needs one byte; size changes on every merge and tells you each set's size. And how the three compression policies differ within a single find. Changing one line, `x = g` to `x = p`x, silently turns path halving into path splitting with identical answers; only a test that checks the exact pointers notices.

What I would do next. Re-run on Linux with `perf stat` to count cache and branch misses directly, since the mechanisms above are inferred, and add a workload whose first endpoint moves sequentially, to test locality properly.

(d) AI use

Tools and how much. I used Claude Code (model Claude Opus 5.5) in VS Code as an agent throughout the project. It wrote nearly all of the code in the final repository: the library, the tests, the benchmark harness, the study runner, the plot script and the side experiments. It also ran the study, analysed the results and drafted this report, which I then reviewed and edited. My own part was choosing the algorithm and track, directing and checking the work, and learning the material well enough to explain it. I relied on it heavily as a tutor.

What I used it for.

- **Implementation:** the policy-based design, the instrumentation, and exception-based edge-case handling.
- **Tests:** the oracle comparison and invariant checks, plus mutation testing (eight injected bugs) to show that the tests catch real errors.
- **Benchmark harness and plots:** workload generators, CPU pinning, repetitions, the drift monitor, and the figures and summary tables.
- **Debugging the measurements:** the three pitfalls in (b) and (c) were each diagnosed with small, targeted experiments.
- **Explaining concepts:** union by rank versus union by size, the $\log_2 n$ height bound, path compression, and the four find policies line by line.
- **Report:** structure and first draft of every section.

Where the AI was wrong, unhelpful or misleading.

#	What the AI produced	How it was caught	What was done
1	A harness that replayed the same op sequence for every timed rep (carried over from the first version), so at $n = 2^{10}$ every variant but naive+none looked 2.5–4.3× faster than it is	ns/op tripled between 2^{10} and 2^{12}, although both sizes fit in L1	A side test confirmed the branch predictor had learned the sequence; the run was stopped, and each rep now gets a fresh sequence (`--replay-same-sequence` reproduces the flaw)
2	A false all-clear on throttling: it compared a foreground run with a background run, found them equal, and concluded nothing was throttled, but both runs were throttled the same way	Later, the same measurement gave 21 or 38 ns/op depending on what ran before it	Traced to Windows power throttling; the harness now opts out, and the run was restarted (`--allow-power-throttling` reproduces it)
3	A side-experiment helper that discarded the checksum, reintroducing the dead-code bug it had documented itself; rank+none looked 35% faster and swapped places with rank+full	Its numbers disagreed with the main benchmark	Ruled out variant order, heap layout and code alignment one at a time; fixed with a `volatile` sink; re-ran every side experiment
4	Overclaims in its draft findings, e.g. "the compression spread shrinks to 2–12% everywhere at 2^{24}" and "full compression is never fastest by a meaningful margin"	Every number was checked against the data before committing	Corrected: binomial stays at 22–25%, and full compression is fastest in 6 of 180 configurations, by up to 5%

It also did not plan for the laptop going to sleep during the hour-long run. Three chunks measured around an 11-hour sleep were about 25% slow and had to be re-measured; the runner now prevents idle sleep and warns when the canary drifts.

What I understood versus what I took on trust. I can explain the data structure and every policy line by line: union by rank versus size, why rank bounds the height by $\log_2 n$, the four find policies, the core invariant and where it is relied on (link must only ever receive two distinct roots), and why the benchmark needs the `volatile` sink. I took the inverse-Ackermann bound itself on trust; I have not worked through Tarjan's proof. I also took on trust the parts of the harness I did not study closely: the sequence pool, the Windows API calls for CPU pinning and power throttling, and the statistics in the plot script. The explanations that rely on cache misses and branch mispredictions are inferences from controlled experiments rather than counter measurements, and I am not fully sure why rank still beat size on Kruskal at 2^{24} but not on random.

References

[1] B. A. Galler and M. J. Fischer. An improved equivalence algorithm. *Communications of the ACM*, 7(5):301–303, 1964.

[2] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[3] R. E. Tarjan and J. van Leeuwen. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2):245–281, 1984.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, chapter "Data Structures for Disjoint Sets". MIT Press.

[5] M. M. A. Patwary, J. Blair and F. Manne. Experiments on union-find algorithms for the disjoint-set data structure. In *Experimental Algorithms (SEA 2010)*, LNCS 6049, pages 411–423. Springer, 2010.

Appendix: reproducing the results

From the repository root, with g++ (or clang) and make; CMake also works (`cmake -B build`, then `cmake --build build --config Release`). The full study takes about 1.5 hours and writes every CSV, `machine.json` and the drift-monitor log into `results/`.

```
make                                  # MSYS2 on Windows: mingw32-make
./build/uf_tests                      # prints: 7315 checks, 0 failures
python scripts/run_study.py --quick   # smoke run, about 1.5 minutes
python scripts/run_study.py           # the full study
python scripts/plot.py results        # figures and summary tables (needs pandas, matplotlib)
make experiments                      # the three side experiments
```